Práctica guiada: Bell, variantes y GHZ

Genesis Silesky

20 de octubre de 2026

1. Práctica guiada: Bell, variantes y GHZ

1.1. Objetivo

Ejecutar el circuito de Bell, predecir resultados antes de verlos, modificarlo para distinguir superposición de determinismo, y generalizarlo a tres qubits (estado GHZ).

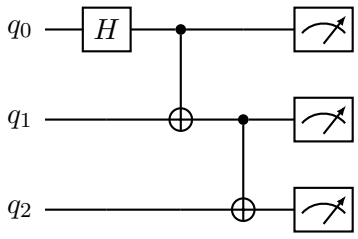
1.2. Conceptos previos

Concepto previo: entrelazamiento y shots

Haber leído el capítulo 05b (estado de Bell) y el 04 (shots y frecuencia).

1.3. Fundamento teórico

- **Bell:** H en q_0 y CNOT $q_0 \to q_1$ dan $\frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$. Se esperan $\approx 50\,\%$ de 00 y $\approx 50\,\%$ de 11.
- **Variante con X:** $X|0\rangle = |1\rangle$ no crea superposición; el CNOT copia ese 1, así que el estado es $|11\rangle$ y el resultado es determinista. Esto aísla el papel de H: es la fuente de la aleatoriedad.
- **GHZ:** con H y dos CNOT en cadena se obtiene $\frac{1}{\sqrt{2}}(|000\rangle + |111\rangle)$.



1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 07 - Practica guiada: Bell, variantes y reto GHZ

# %% Celda 1: importaciones y simulador
import sys

from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.visualization import plot_histogram

```

```python
# La consola de Windows (cp1252) no imprime el dibujo del circuito sin esto.
if hasattr(sys.stdout, "reconfigure"):
    sys.stdout.reconfigure(encoding="utf-8")

sim = AerSimulator()  # un solo simulador reutilizado en todas las celdas

# %% Celda 2: circuito de Bell
qc = QuantumCircuit(2, 2)   # 2 qubits y 2 bits clasicos
qc.h(0)                     # superposicion en el qubit 0
qc.cx(0, 1)                 # CNOT: entrelaza el qubit 1 con el 0
qc.measure([0, 1], [0, 1])  # qubit i -> bit clasico i
print(qc.draw())
# Esperado: ~50% '00' y ~50% '11', y casi nunca '01' ni '10'.
print(sim.run(qc, shots=1000).result().get_counts())

# %% Celda 3: cambiar h(0) por x(0)
# X es el NOT: lleva |0> a |1> sin crear superposicion. El CNOT copia
# ese 1 al qubit 1, asi que el resultado es determinista: siempre '11'.
# Moraleja: el aleatorio de la celda 2 venia de la superposicion de H.
qx = QuantumCircuit(2, 2)
qx.x(0)
qx.cx(0, 1)
qx.measure([0, 1], [0, 1])
print(sim.run(qx, shots=1000).result().get_counts())

# %% Celda 4: reto - estado GHZ de 3 qubits
# Generaliza Bell: (|000> + |111>)/sqrt(2). Cada CNOT "propaga" el valor
# del qubit anterior al siguiente.
ghz = QuantumCircuit(3, 3)
ghz.h(0)
ghz.cx(0, 1)
ghz.cx(1, 2)
ghz.measure([0, 1, 2], [0, 1, 2])
print(sim.run(ghz, shots=1000).result().get_counts())  # solo '000' y '111'

# %% Celda 5: reto - efecto del numero de shots
# Con pocos shots el histograma es muy irregular (ej. 8 contra 2);
# con muchos se acerca al 50/50 teorico. Es ruido estadistico, no fisico.
for shots in [10, 100, 10000]:
    print(shots, sim.run(qc, shots=shots).result().get_counts())
plot_histogram(sim.run(qc, shots=10000).result().get_counts())
```

1.5. Explicación celda por celda

Celda 2 Bell con mediciones; `measure([0,1],[0,1])` conecta cada qubit con un bit clásico.

Celda 3 Sustituye `h(0)` por `x(0)`.

Celda 4 Reto: GHZ de 3 qubits.

Celda 5 Reto: distintos números de shots.

1.6. Salida obtenida

```
{'11': 506, '00': 494}
{'11': 1000}
{'000': 503, '111': 497}
10 {'11': 8, '00': 2}
100 {'00': 51, '11': 49}
10000 {'00': 5003, '11': 4997}
```

(El dibujo de texto del circuito, que imprime el script, no se reproduce aquí.)

1.7. Interpretación

- Con x en lugar de h solo sale 11: ya no hay aleatoriedad.
- GHZ solo da 000 y 111.
- Con 10 shots, las proporciones pueden alejarse mucho de 50/50 (aquí, 8 contra 2); con 10 000 se acercan. Es ruido *estadístico*, no físico (el siguiente capítulo introduce el ruido físico).

Pregunta clave: ¿por qué nunca sale 01 ni 10? Porque el CNOT fuerza que q_1 copie el valor de q_0 en la base computacional.

1.8. Ejercicios

1. Prediga y verifique qué ocurre con $h(0)$ seguido de $x(1)$ (sin CNOT).
2. Extienda GHZ a 5 qubits.
3. Mida solo uno de los tres qubits de GHZ y observe cuál es la distribución del resto.